TECNOLÓGICO DE SOFTWARE · TSU EN DESARROLLO E INNOVACIÓN DE SOFTWARE

MONGODB

Guardar documentos cuando las filas de MySQL no son la forma natural de los datos

REPORTE · UNIDAD II · EXPOSICIÓN · UNA HERRAMIENTA DE BACKEND



EQUIPO 5

Ariff Iazid Medina Gómez

Joaquin Uriona

Jesus Fernando Castro Hernandez

Mateo Emanuel Martin Tec

Fundamentos de Arquitectura de Software y Desarrollo Backend

26 de octubre de 2026

1. QUÉ PROBLEMA RESUELVE

Durante el curso guardamos la información de la API en MySQL: definimos tablas, columnas y tipos antes de insertar la primera fila, y relacionamos las tablas con llaves foráneas y `JOIN`. Ese modelo funciona muy bien cuando todos los registros tienen la misma forma. El problema aparece cuando no la tienen.

Pensemos en el catálogo de una tienda. Una playera tiene talla, una lámpara tiene voltaje y un libro tiene número de páginas y un autor. En una tabla `productos` hay dos salidas, y las dos son incómodas. La primera es agregar una columna por cada atributo posible: con tres productos ya quedan seis celdas en `NULL` de quince, y cada tipo nuevo de producto obliga a ejecutar un `ALTER TABLE` que afecta a todas las filas. La segunda es crear una tabla por tipo de producto, lo que multiplica las tablas y obliga a unirlas cada vez que se quiere listar el catálogo completo.

MongoDB es una base de datos NoSQL orientada a documentos. En lugar de filas guarda **documentos** con una estructura parecida a JSON (internamente BSON), agrupados en **colecciones**. Cada documento lleva solo los campos que necesita, así que la playera, la lámpara y el libro pueden vivir en la misma colección sin columnas vacías. Además, un documento puede contener objetos y arreglos, de modo que el autor del libro puede ir *dentro* del documento del libro.

La idea central para modelar en MongoDB la resume su documentación: los datos que se consultan juntos deben guardarse juntos. Esto da dos herramientas:

- **Embeber**: si siempre se leen juntos, un dato va dentro del otro. El departamento dentro del empleado, el autor dentro del libro. Se obtiene todo en una sola consulta, sin `JOIN`.
- **Referenciar**: si algo crece sin límite o se consulta por separado, se guarda en otra colección y se enlaza por su identificador. Por ejemplo, un producto puede guardar sus cinco reseñas más recientes y dejar el historial completo en una colección `resenas`.

2. LAS PIEZAS CON LAS QUE SE TRABAJA

Pieza	Qué es	En nuestro ejemplo
Servidor	El proceso `mongod` que guarda los datos. Puede correr en la nube (MongoDB Atlas) o localmente.	Un contenedor `mongo:8` levantado con Docker Compose, o un cluster gratuito de Atlas.
Cliente	La aplicación que hace las consultas.	Una API REST en Node.js con Express.
Driver	La librería oficial que traduce el código del cliente en operaciones de la base.	El paquete `mongodb` de npm (versión 7).
Archivo de configuración	Guarda la cadena de conexión fuera del código.	Variable `MONGO_URI`, en un archivo `.env` excluido de Git.
Servicio en la nube	MongoDB Atlas: aloja y administra el servidor, con usuarios, red y respaldos.	Opcional; basta con cambiar `MONGO_URI`.

Dentro del servidor la información se organiza en tres niveles: una **base de datos** (`tienda`) contiene **colecciones** (`productos`) y cada colección contiene **documentos**. Cada documento recibe automáticamente un campo `_id` único de tipo `ObjectId`.

3. CÓMO SE USA, EN PASOS

Siguiendo la guía oficial *Get Started with the Node.js Driver*, los pasos con Atlas son:

1. **Crear un cluster gratuito** en MongoDB Atlas.
2. **Crear un usuario de base de datos** y guardar su contraseña. Este usuario es distinto de los usuarios de nuestra aplicación.
3. **Obtener la cadena de conexión** desde *Connect → Drivers → Node.js* y reemplazar `<db_password>` por la contraseña real. La cadena tiene la forma `mongodb+srv://<usuario>:<db_password>@<cluster>.mongodb.net/`.
4. **Instalar el driver** con `npm install mongodb`. Requiere Node.js 16.20.1 o superior.
5. **Conectar y operar**: crear un `MongoClient` con la cadena, elegir base y colección, y usar `insertOne`, `find`, `findOne`, `updateOne` y `deleteOne`.

Si no se quiere usar la nube, los pasos 1 a 3 se sustituyen por `docker compose up -d` con la imagen oficial `mongo` y la cadena `mongodb://localhost:27017`. El resto del código no cambia: solo cambia el valor de `MONGO_URI`.

4. EN QUÉ SE PARECE Y EN QUÉ SE DIFERENCIA DE LO VISTO EN EL CURSO

Frente a MySQL. Es la comparación directa, porque ambos guardan y consultan datos. La equivalencia de conceptos ayuda a no verlo como algo totalmente nuevo:

MySQL	MongoDB
tabla	colección
fila	documento
columna	campo
`WHERE precio <= 350`	`{ precio: { $lte: 350 } }`
`JOIN`	embeber el dato, o `$lookup` en una agregación
`ALTER TABLE` para agregar una columna	`$set` agrega el campo solo en el documento que lo necesita
esquema obligatorio	validación opcional con `$jsonSchema`

La diferencia de fondo es dónde vive el esquema. En MySQL la base lo exige; en MongoDB, por defecto, lo exige la aplicación.

Frente a Docker. No compiten: Docker ejecuta procesos en contenedores y MongoDB guarda datos. Se combinan de forma natural; en nuestra demo el servidor de MongoDB corre dentro de un contenedor, igual que hicimos con MySQL.

Frente a GraphQL. Tampoco compiten. GraphQL define el contrato entre el cliente y la API; MongoDB es donde la API busca los datos. Un resolver de GraphQL puede ejecutar el mismo `findOne` que usamos en REST.

Frente al token y los permisos. Son dos capas de seguridad distintas. La API valida el token del usuario final y decide qué puede hacer; MongoDB exige su propio usuario y contraseña, con roles por base de datos, para aceptar la conexión de la API. Un buen diseño da al usuario de la API solo el rol `readWrite` sobre su base, siguiendo el principio de mínimo privilegio. Las credenciales viven en la cadena de conexión, por eso nunca se escriben en el código ni se suben a GitHub.

5. EJEMPLO: UNA API DE PRODUCTOS

Construimos una API pequeña, como las del curso, con el CRUD completo de `/productos` (`POST`, `GET`, `PATCH`, `DELETE`) y un filtro `?maxPrecio=`. El código está en la carpeta `demo/` del repositorio; la parte

esencial es:

```
const client = new MongoClient(process.env.MONGO_URI);
const productos = client.db('tienda').collection('productos');

app.get('/productos/:sku', async (req, res) => {
  const p = await productos.findOne({ sku: req.params.sku });
  p ? res.json(p) : res.sendStatus(404);
});
```

Tres decisiones merecen explicación. Primero, la cadena de conexión se lee de una variable de entorno. Segundo, el filtro de `findOne` es un objeto de JavaScript y no una cadena de texto; no hay SQL que concatenar, lo que reduce el riesgo de inyección, aunque igual validamos la entrada. Tercero, el resultado ya es un objeto: se envía con `res.json` sin mapear columnas a propiedades.

Al ejecutar la demo (`npm run demo`) se insertan tres productos con formas distintas y se obtienen estos resultados:

Petición	Resultado
`POST` playera `{ talla: "M" }`, lámpara `{ voltaje: "127 V" }`, libro `{ paginas: 280, autor: { … } }`	`201` en los tres; conviven en la misma colección.
`GET /productos?maxPrecio=350`	`200` con la playera y el libro (filtro `$lte`).
`PATCH /productos/PLY-01` con `{ colores: ["negro", "blanco"] }`	`200`; el campo nuevo aparece solo en la playera, sin migración.
`POST` sin `precio`	`400`; la validación la hace la API.
`POST` con un `sku` repetido	`409`; lo impide un índice único sobre `sku`.

Los dos últimos casos muestran algo importante: la flexibilidad no significa ausencia de reglas. Las reglas que en MySQL daba la tabla (columna obligatoria, llave única) aquí las pusimos nosotros, con validación en la API y un índice único.

6. QUÉ GANA EL EQUIPO Y QUÉ COSTO O LÍMITE TIENE

Lo que gana:

- **Un modelo que cambia con la aplicación.** Agregar un atributo a un tipo de producto no obliga a migrar toda la colección.
- **Lecturas más simples.** Cuando se modela según cómo se leen los datos, una pantalla completa sale de una sola consulta, sin `JOIN`.
- **Continuidad con JavaScript.** Los documentos se manipulan como objetos, desde la consulta hasta la respuesta JSON.
- **Servidor administrado.** Con Atlas el equipo no instala ni mantiene el servidor; la API solo necesita la cadena de conexión.

Lo que cuesta:

- **Diseño previo distinto.** En relacional se normaliza y después se consulta; en MongoDB hay que pensar primero qué consultas hará la aplicación. Un mal modelo, como embeber arreglos que crecen sin límite, se paga después en rendimiento.
- **Disciplina.** La validación de esquema es opcional: si nadie la define, nadie la exige, y con el tiempo pueden convivir documentos inconsistentes.
- **Datos duplicados.** Embeber implica a veces copiar información (por ejemplo, el nombre del autor en varios libros) y mantener esas copias al día.
- **Límites del plan gratuito.** Según la documentación de Atlas, el cluster gratuito ofrece 0.5 GB de almacenamiento, 100 operaciones por segundo y un máximo de 500 conexiones. Es suficiente para aprender y para prototipos, pero no para producción. Corregimos aquí un dato del borrador: el plan gratuito sí permite activar respaldos, con una instantánea diaria y hasta cuatro guardadas.

7. CUÁNDO NO CONVIENE USARLA

- **Cuando los datos son muy relacionales.** Un sistema escolar con alumnos, materias, grupos, profesores y calificaciones necesita cruzar entidades en casi cada consulta. Ahí las llaves foráneas y los `JOIN` de un modelo relacional son la herramienta natural.
- **Cuando la estructura debe ser rígida.** Si todos los registros deben cumplir exactamente las mismas reglas y la base debe garantizarlo, una tabla con restricciones lo hace por defecto.
- **Cuando las transacciones son el núcleo.** MongoDB sí soporta transacciones con varios documentos, pero si el corazón del sistema son operaciones que modifican muchos registros a la vez, como pagos o inventarios contables, una base relacional ofrece ese comportamiento como caso principal y no como excepción.

La pregunta útil no es «¿SQL o NoSQL?», sino «¿qué forma tienen mis datos y cómo los lee mi API?». Muchos sistemas reales usan ambas: MySQL para la parte transaccional y MongoDB para catálogos, perfiles o registros de eventos.

8. CONCLUSIÓN

MongoDB no reemplaza a MySQL; resuelve un problema distinto. Conviene cuando los registros tienen formas variables o cuando la aplicación lee objetos completos, y se integra con lo que ya construimos en el curso: corre en Docker, sirve como fuente de datos para REST o GraphQL y agrega una capa más de control de acceso. Su flexibilidad traslada responsabilidades al equipo, que debe modelar según las consultas y definir sus propias reglas de validación.

9. FUENTES CONSULTADAS

1. MongoDB. *Get Started with the Node.js Driver*. MongoDB Documentation. https://www.mongodb.com/docs/drivers/node/current/get-started/
2. MongoDB. *Data Modeling*. MongoDB Database Manual. https://www.mongodb.com/docs/manual/data-modeling/
3. MongoDB University. *Connecting to MongoDB in Node.js*. https://learn.mongodb.com/courses/connecting-to-mongodb-in-nodejs
4. MongoDB. *Atlas Free Cluster Limits*. MongoDB Atlas Documentation. https://www.mongodb.com/docs/atlas/reference/free-shared-limitations/ (consultado el 25 de octubre de 2026)